Fontys

› FOR SOCIETY

SOUND SOURCE LOCALIZATION

MESSAGING PROTOCOL : UART

Casey Stijlaart

2024-05-29

VERSION HISTORY

Version	Date	Author	Changes
1.0	28-05-2024	Casey Stijlaart	First Version, base with teensy and indication PCB

INTRODUCTION

This document serves as a guide for the UART messaging protocol. It documents all the possible messages per destination.

As within the system, messages will be sent via the communication protocol. To ensure a proper communication rules and standards need to be set. Therefore, to create a proper messaging protocol, there has to be looked at trough what type of communication it is sent. Within a send data packet, there is made use of multiple bytes. Each Byte with its specification on what kind of data it will carry.

TABLE OF CONTENTS

MESSAGE STRUCTURE

Each message follows the same rules regarding its format. The message format consists of the following components:

Field	Size (bytes)	Endianness	Description
Start Bytes	2	N/A	Unique identifier for the start of the message (ASCII characters 'S', 'L').
Payload Size	1	Big-endian	Size of the payload, represented as an unsigned integer.
Command	X	N/A	Variable-length payload, where X is the value indicated by the payload size field.
CRC	1	Big-endian	Cyclic Redundancy Check the value of the message.

Table 1: The message structure

The format of the message can be described as follows, divided per message part:

START BYTES

The message starts with 2 bytes that form the “start bytes” of the message. In this case, the start bytes are the ASCII characters ‘S’ and ‘L’. These bytes serve as a unique identifier for the start of the message and allow the receiver to detect the beginning of a new message.

PAYLOAD SIZE

Following the start bytes, 1 byte indicate the size of the payload. The size is represented as an unsigned integer and is encoded in big-endian format. This means that the most significant byte appears first in the message.

PAYLOAD

After the payload size field, the message contains a variable-length payload of X amount of bytes, where X is the value indicated by the payload size field. The content of the payload is application-specific and can include any data or information that needs to be communicated between the sender and receiver.

CRC

Finally, the message ends with a 1-byte CRC (*Cyclic Redundancy Check*) value that provides a checksum for the message. The CRC is used to detect any errors that may have occurred during transmission and can help ensure message integrity. Like the payload size field, the CRC value is encoded in big-endian format. The used CRC is the CRC-8/CCITT, as it is a flexible type and can be used for many different implementation. The CRC is is calculated over the payload, including the command id. By adhering to this format, both the sender and receiver can reliably communicate with each other and ensure the integrity of the messages being exchanged.

COMMAND API

Different commands can be sent and requested, dependings on its source and destination, these are defined in the API (*Application Programming Interface*). To ensure clearity, they are seperated by their destination, as they will always connected to the ESP32 for the purpose of the project.

The defined commands are as follows:

TEENSY

ID	Name	Parameters	Return Values	Description
0x01	GetDeviceId	N/A	128-bit Device ID	The universally unique device ID
0x02	GetDeviceStatus	N/A	8-bit Status structure	Request the current device status
0x03	GetDeviceType	N/A	8-bit Device structure	Request the device type
0x04	GetNrMics	N/A	An 8-bit integer from 0 to 8	Gets the number of connected microphones
0x05	GetAngle	N/A	An 8-bit integer from 0 to 100	Gets the current angle of the latest the detected sound source.
0x06	GiveAngle	N/A	An 8-bit integer from 0 to 100	Sends the current angle of the current measured detected sound source.
0x07	GetDecibel	N/A	An 8-bit integer from 0 to 100	Gets the current decibels of the latest the detected sound source.
0x08	GiveDecibel	N/A	An 8-bit integer from 0 to 100	Sends the current decibels of the current measured detected sound source.
0x09	GiveData	N/A	Two times 8-bit integer from 0 to 100	Sends the current decibels and angle of the current measured detected sound source.
0x10	AddMic	N/A	An uint8_t 0 or 1, depending on ret val.	Request to add a microphone
0x11	RemoveMic	8-bit unsigned int with mic id	An uint8_t 0 or 1, depending on return val.	Request to remove the given microphone
0xEE	Error	8-bit error type	N/A	The error response

Table 2: Command API for the teensy module

INDICATION

ID	Name	Parameters	Return Values	Description
0x01	GetDeviceId	N/A	128-bit Device ID	The universally unique device ID
0x02	GetDeviceStatus	N/A	8-bit Status structure	Request the current device status
0x03	GetDeviceType	N/A	8-bit Device structure	Request the device type
0x04	SetIndication	N/A	8-bit Indication structure	Request to set the indication.
0xEE	Error	8-bit error type	N/A	The error response

Table 3: Command API for the Indication module

ERROR TYPES

The given error types are relevant for

ID	Name	Description
0x01	InvalidCommand	The received command is not an valid option for the receiving device.
0x02	Busy	The given device is currently performing a different task.
0x03	Unsupported	The request data is not supported by the given device.
0x04	Device	The device has an internal error.
0x05	Microphones	The amount of connected microphones is not valid.

Table 4: The error types that could be send back upon a message.

RESPONSE

There are two possible responses that can be given upon request. These are the success and the error return. The response of a device with success is the command id ^ 0x80 and the additional return values. The response of a command with failure is 0xEE with as return the byte with the error type.

EXAMPLES

In the given example the GetNrMics is requested and the two possible responses are given.

Index	Value	Description
0	'S'	First start character -> start[0]
1	'L'	Second start character -> start[1]
2	0x01	The byte of payload size
3	0x03	The first byte of payload -> command.id
4	0x09	The byte of uint8_t CRC

Table 5: Example scenario of requesting the current connected microphones

Index	Value	Description
0	'S'	First start character -> start[0]
1	'L'	Second start character -> start[1]
2	0x02	The byte of payload size
3	0x83	The first byte of payload -> command.id
4	0x02	The byte with the data -> command.data[0]
5	0x87	The byte of uint8_t CRC

Table 6: Succes return message when requesting the current connected microphones

Index	Value	Description
0	'S'	First start character -> start[0]
1	'L'	Second start character -> start[1]
2	0x02	The byte of payload size
3	0xEE	The first byte of payload -> command.id
4	0x05	The byte with the error type -> command.data[0], Not enough connected microphone
5	0x8E	The byte of uint8_t CRC

Table 7: Error return message when requesting the current connected microphones

GLOSSARY

***API* – Application Programming Interface**: A software intermediary that allows two applications to talk to each other 2

***CRC* – Cyclic Redundancy Check**: An error detection technique using a polynomial to generate a series of two 8-bit block check characters that represent the entire block of data. 1

CRC-8/CCITT: A fast error detection algorithm, based on a 8-bit CRC with polynomial x^8+x^2+x+1. 1